MEMORIA DE PROYECTO: OPTIMIZACIÓN Y REFACTORIZACIÓN

1. Gestión del Repositorio.

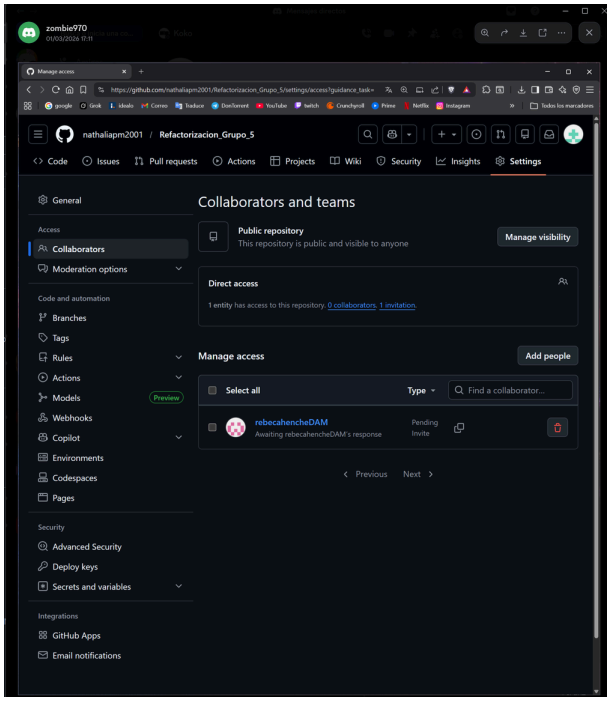
Creación del repositorio:

Uso de .gitignore: Se ha incluido un archivo .gitignore para evitar subir archivos innecesarios del IDE.

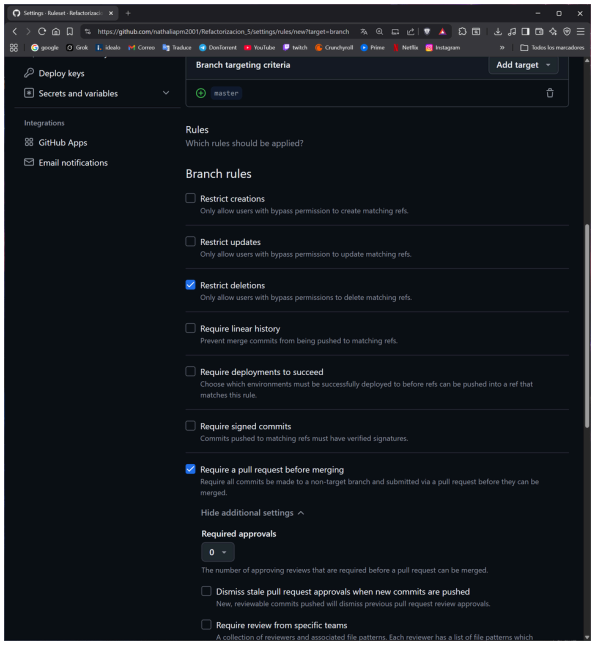
Uso de README.md : Se ha incluido un archivo README.md es el documento que se muestra automáticamente en la página principal del repositorio, ara que cualquier persona entienda de qué trata el código.

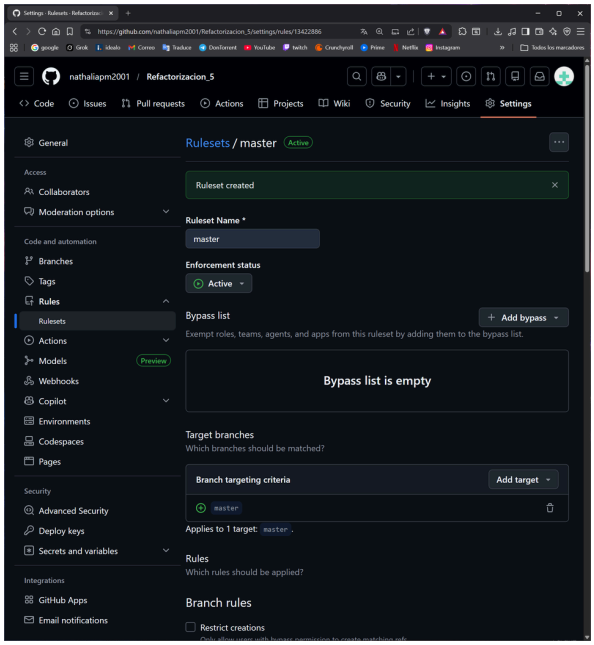
Estrategia de Ramas: Se ha empleado una rama develop como base de integración y ramas temporales de tipo feature/nombre-tarea para cada refactorización, garantizando que la rama principal siempre sea estable.

Añadir colaboradores:



Rama Master Protegida: Se ha configurado una regla de protección en la rama master para impedir "pushes" directos, obligando al uso de **Pull Requests** y revisiones de código.





2. Refactorizaciones Realizadas.

Extracción de Constantes en clase Manager.

Descripción: Se han sustituido "números mágicos" por constantes con nombre, como MESES_DEL_ANIO = 12 y BONO_POR_EMPLEADO = 1000.

Justificación: Mejora drásticamente la **legibilidad** y el **mantenimiento**. Si en el futuro los meses contables o el bono cambian, solo hay que modificar un único punto del código en lugar de buscar el número por todo el proyecto.

Herramienta IDE: Refactor > Extract Constant....

Foto de código antes de la extracción.

Employee.java | Manager.java

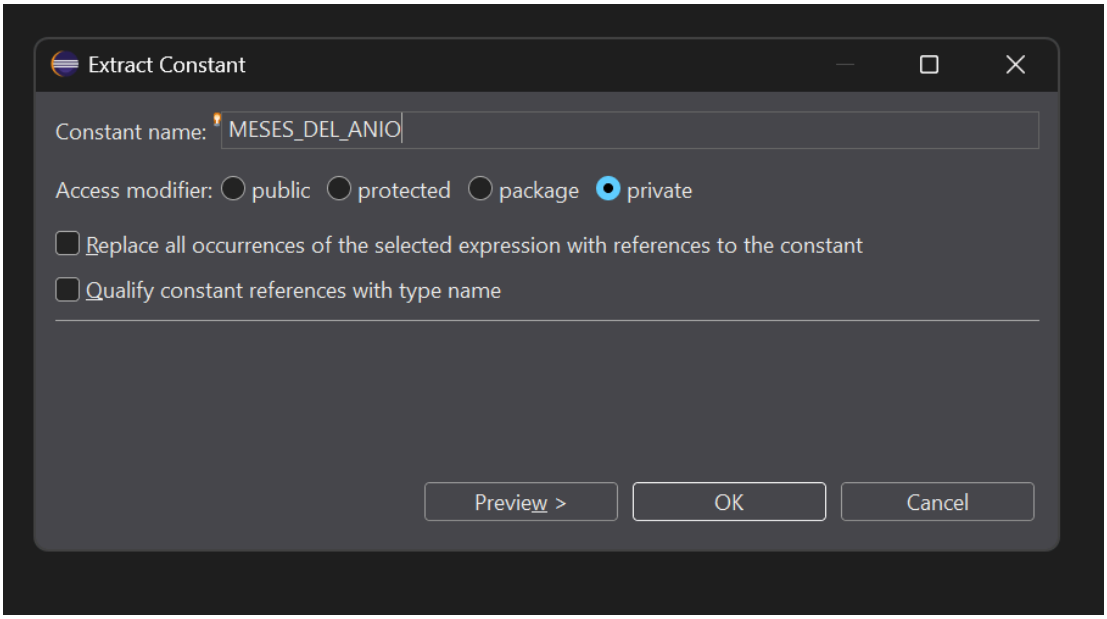
```java
package com.refactoring.projects.project03;

public class Manager {

    public String nombre;
    public double salario;
    public String departamento;
    public int empleadosACargo;

    public void calcularSalarioAnual() {
        System.out.println(salario * 12 + empleadosACargo * 1000);
    }
}
```

Foto usando la refactorización de extraer constantes.



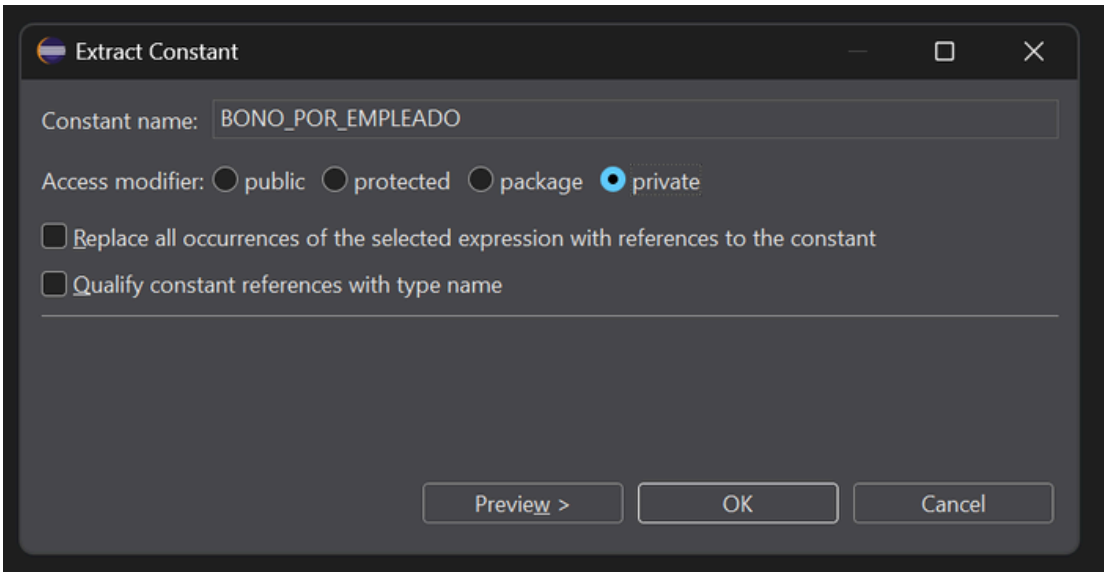
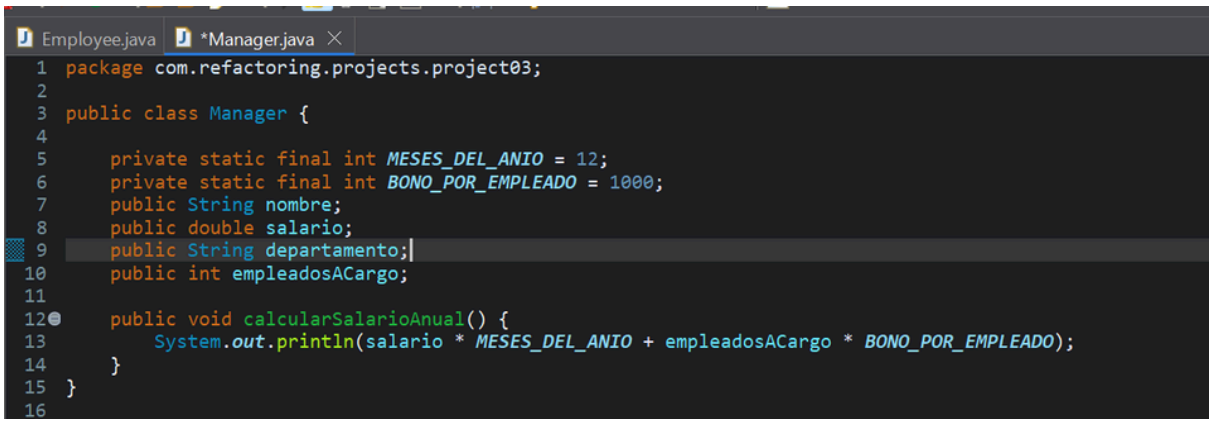


Foto resultado del uso del método.



Extracción de Constantes en clase Employee.

Descripción: Se han sustituido "números mágicos" por constantes con nombre, como MESES_DEL_ANIO = 12.

Justificación: Mejora drásticamente la **legibilidad** y el **mantenimiento**. Si en el futuro los meses contables, solo hay que modificar un único punto del código en lugar de buscar el número por todo el proyecto.

Herramienta IDE: Refactor > Extract Constant....

Foto de código antes de la extracción.

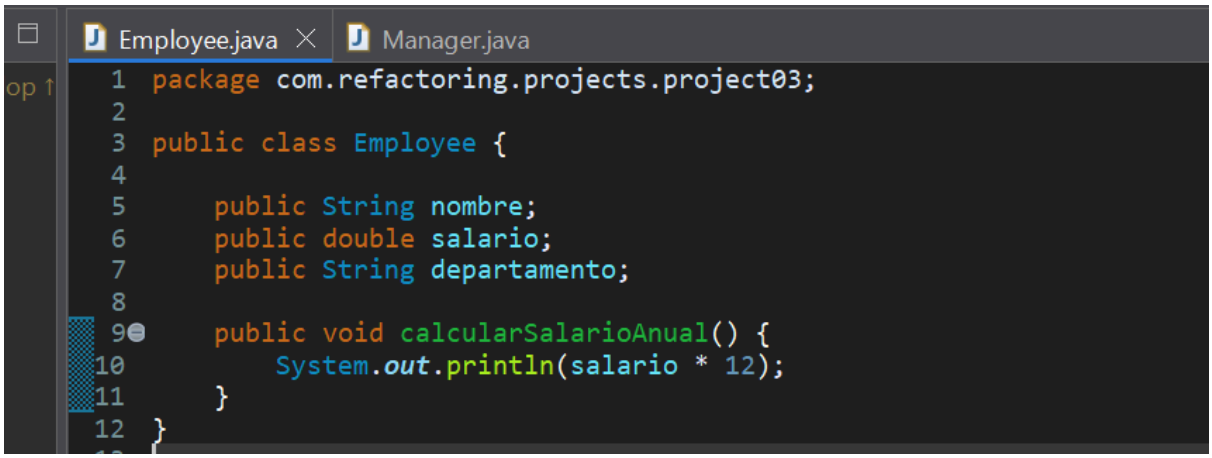


Foto usando la refactorización de extraer constantes.

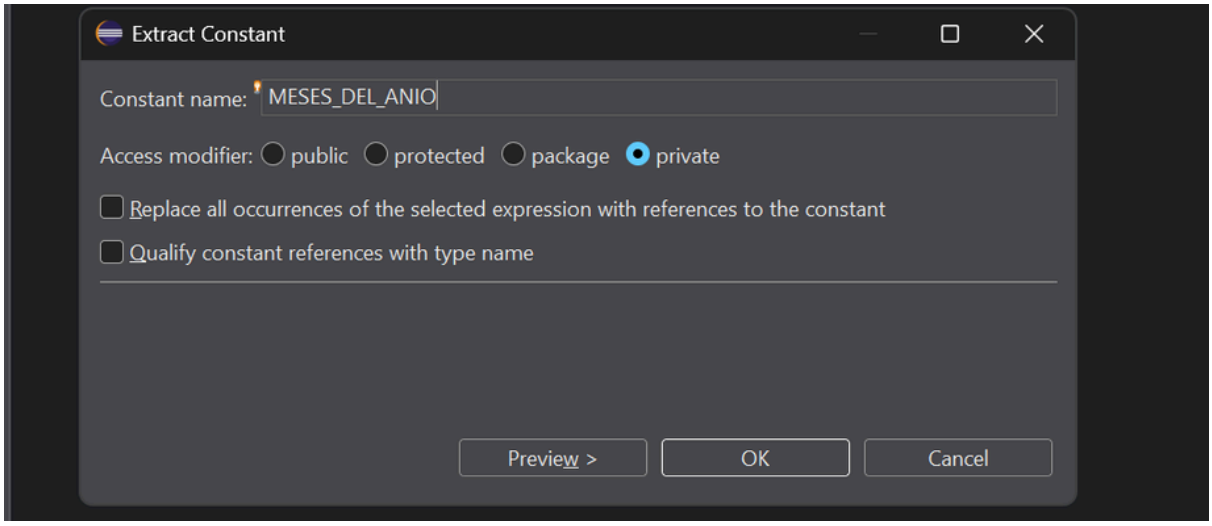


Foto resultado del uso del método.

```java
package com.refactoring.projects.project03;

public class Employee {

    private static final int MESES_DEL_ANIO = 12;
    public String nombre;
    public double salario;
    public String departamento;

    public void calcularSalarioAnual() {
        System.out.println(salario * MESES_DEL_ANIO);
    }
}
```

Encapsulación de Campos.

Encapsulación de Campos en la clase Employee.

Descripción: Se han cambiado los modificadores de acceso de los atributos de nombre, salario y departamento de public a private.

Justificación: Aplicamos el principio de ocultación de datos. Al hacer los atributos privados y usar *getters* y *setters*, protegemos la integridad de los datos de la clase y permitimos validaciones futuras sin romper el código externo.

Herramienta IDE: Refactor > Encapsulate Field....

Foto de código antes de la encapsulación.

```java
package com.refactoring.projects.project03;

public class Employee {

    private static final int MESES_DEL_ANIO = 12;
    public String nombre;
    public double salario;
    public String departamento;

    public void calcularSalarioAnual() {
        System.out.println(salario * MESES_DEL_ANIO);
    }
}
```

Foto usando la refactorización de encapsulación.

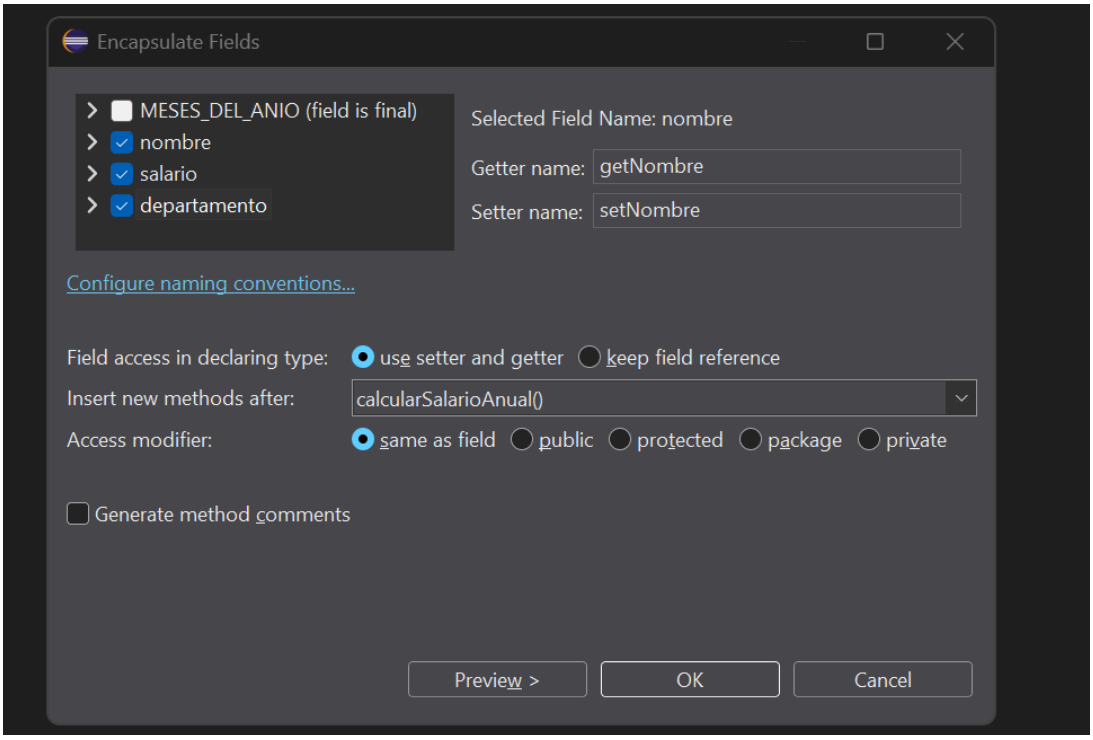
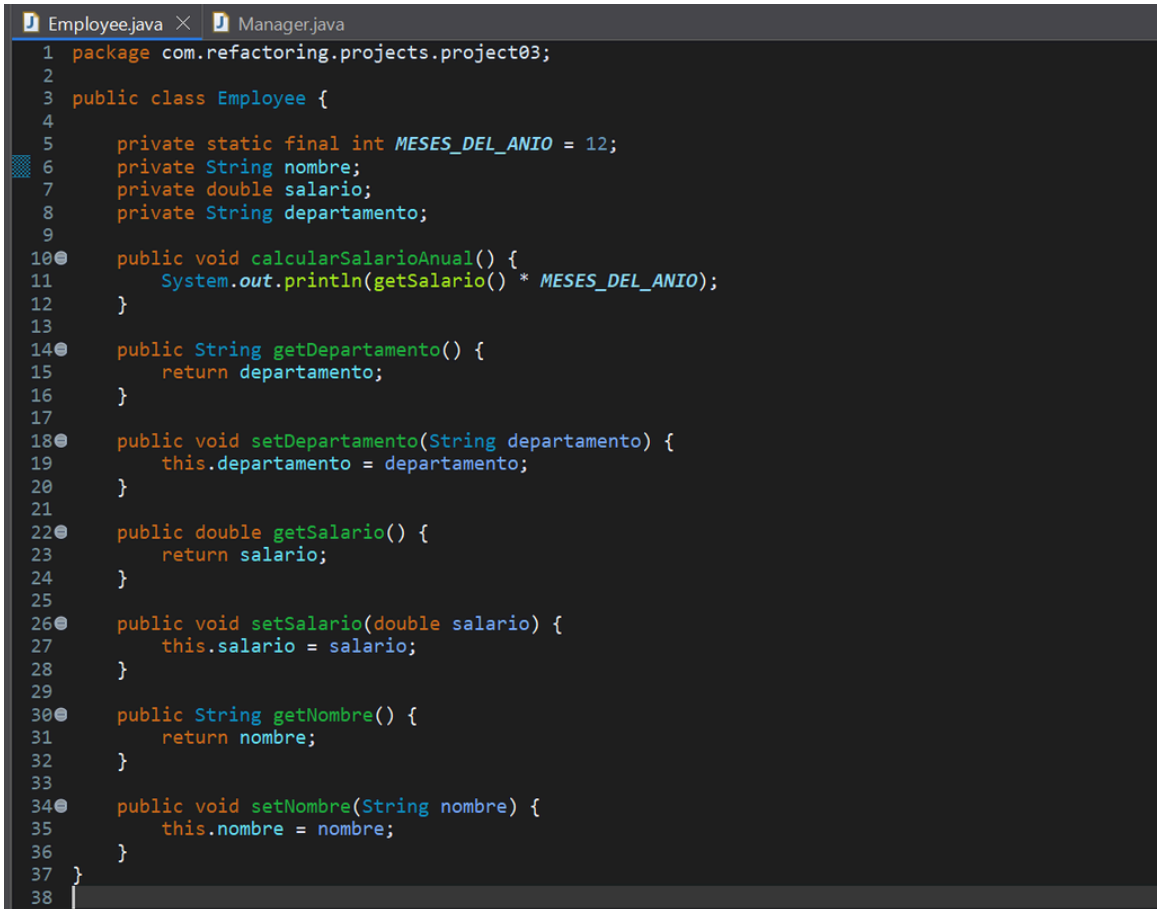


Foto resultado del uso del método.



Employee.java | Manager.java

```java
package com.refactoring.projects.project03;

public class Employee {

    private static final int MESES_DEL_ANIO = 12;
    private String nombre;
    private double salario;
    private String departamento;

    public void calcularSalarioAnual() {
        System.out.println(getSalario() * MESES_DEL_ANIO);
    }

    public String getDepartamento() {
        return departamento;
    }

    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }

    public double getSalario() {
        return salario;
    }

    public void setSalario(double salario) {
        this.salario = salario;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Encapsulación de Campos en la clase Manager.

Descripción: Se han cambiado los modificadores de acceso de los atributos de nombre, salario, departamento y empleadosACargo de public a private.

Justificación: Aplicamos el principio de ocultación de datos. Al hacer los atributos privados y usar *getters* y *setters*, protegemos la integridad de los datos de la clase y permitimos validaciones futuras sin romper el código externo.

Herramienta IDE: Refactor > Encapsulate Field....

Foto de código antes de la encapsulación.

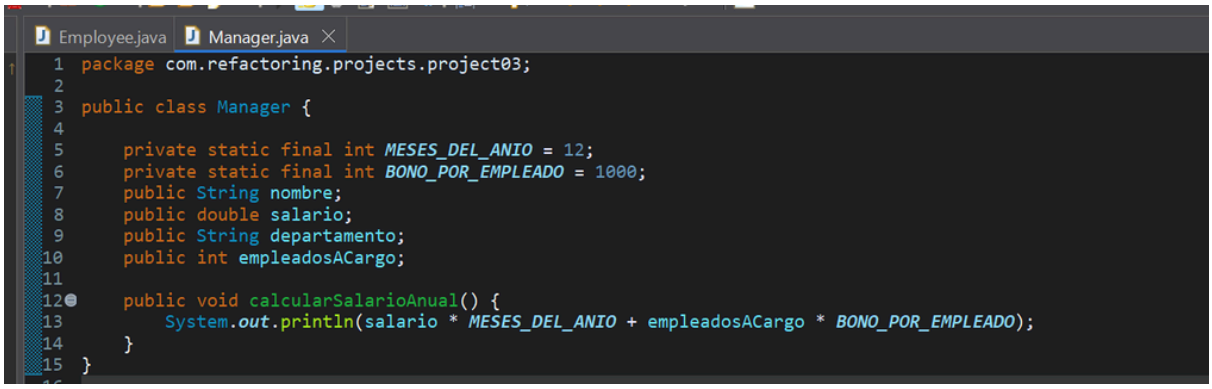


Foto usando la refactorización de encapsulación.

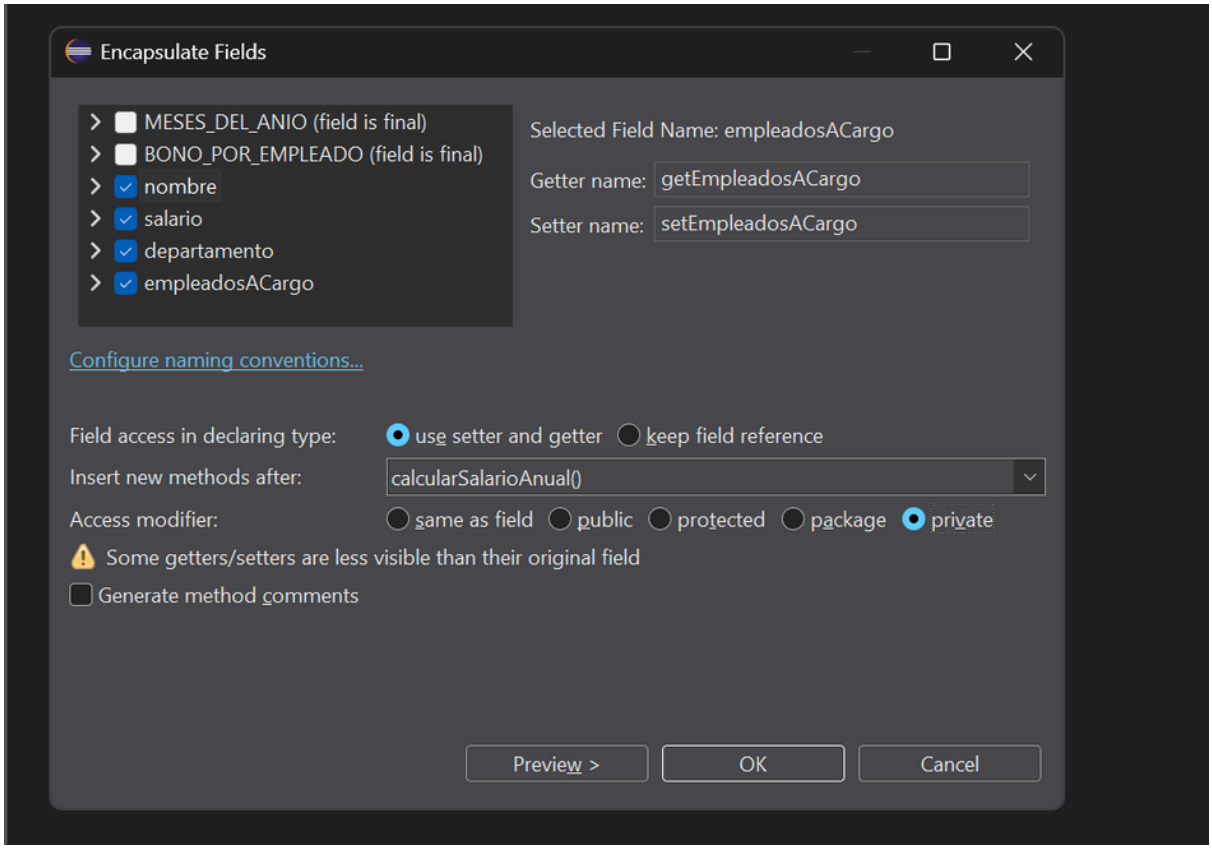


Foto resultado del uso del método.

```java
package com.refactoring.projects.project03;

public class Manager {

    private static final int MESES_DEL_ANIO = 12;
    private static final int BONO_POR_EMPLEADO = 1000;
    private String nombre;
    private double salario;
    private String departamento;
    private int empleadosACargo;

    public void calcularSalarioAnual() {
        System.out.println(getSalario() * MESES_DEL_ANIO + getEmpleadosACargo() * BONO_POR_EMPLEADO);
    }

    private int getEmpleadosACargo() {
        return empleadosACargo;
    }

    private void setEmpleadosACargo(int empleadosACargo) {
        this.empleadosACargo = empleadosACargo;
    }

    private String getDepartamento() {
        return departamento;
    }

    private void setDepartamento(String departamento) {
        this.departamento = departamento;
    }

    private double getSalario() {
        return salario;
    }

    private void setSalario(double salario) {
        this.salario = salario;
    }

    private String getNombre() {
        return nombre;
    }

    private void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Extracción de métodos.

Extracción de métodos en la clase Manager.

Descripción: Se han cambiado un fragmento de código dentro del método calcularSalarioAnual() que realizaba un cálculo específico (el cálculo del bono por empleados a cargo). Ese fragmento se ha extraído para crear un nuevo método independiente.

Justificación: Al darle un nombre descriptivo (calcularBono), el código se explica por sí mismo. Ya no hace falta descifrar qué hace la operación matemática. Si en el futuro la empresa cambia la lógica del bono, solo se tendrá que modificar el código en el método calcularBono(), sin tocar la estructura principal de calcularSalarioAnual().

Herramienta IDE: Refactor > Extract Method....

Foto de código antes de la extracción de método.

```java
package com.refactoring.projects.project03;

public class Manager {

    private static final int MESES_DEL_ANIO = 12;
    private static final int BONO_POR_EMPLEADO = 1000;
    private String nombre;
    private double salario;
    private String departamento;
    private int empleadosACargo;

    public void calcularSalarioAnual() {
        System.out.println(getSalario() * MESES_DEL_ANIO + getEmpleadosACargo() * BONO_POR_EMPLEADO);
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public double getSalario() {
        return salario;
    }

    public void setSalario(double salario) {
        this.salario = salario;
    }

    public String getDepartamento() {
        return departamento;
    }

    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }

    public int getEmpleadosACargo() {
        return empleadosACargo;
    }

    public void setEmpleadosACargo(int empleadosACargo) {
        this.empleadosACargo = empleadosACargo;
    }


}
```

Foto usando la refactorización de extracción de método.

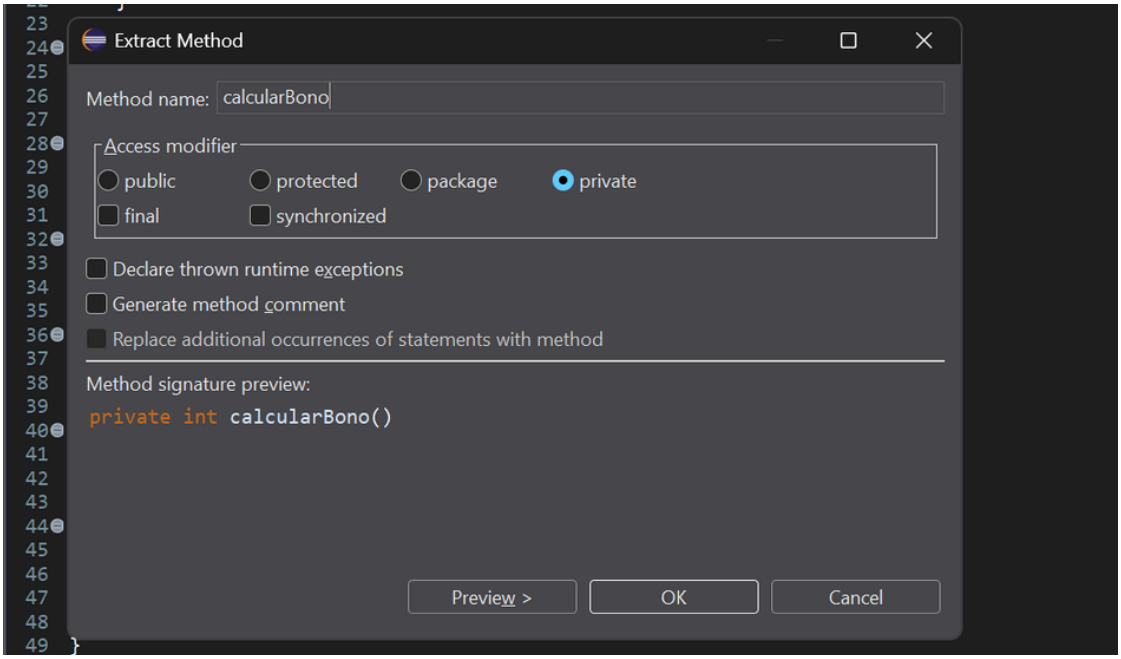


Foto resultado del uso del método.

```java
public void calcularSalarioAnual() {
    System.out.println(getSalario() * MESES_DEL_ANIO + calcularBono());
}

private int calcularBono() {
    return getEmpleadosACargo() * BONO_POR_EMPLEADO;
}
```

Aplicación del Principio de Responsabilidad Única.

Responsabilidad Única en la clase Manager y Employee.

Descripción: Se ha extraído la lógica de impresión y salida por consola de los métodos calcularSalarioAnual() hacia una nueva clase denominada EmployeeManager.

Justificación: Las clases de entidad (Employee y Manager) deben gestionar únicamente lógica de negocio y datos, no la interacción con el usuario o la interfaz. Al centralizar la salida en EmployeeManager, el sistema es más escalable y fácil de testear.

Herramienta IDE: Menú Refactor > Change Method y posterior movimiento de lógica a la nueva clase.

Foto de código antes de mover de clase en Manager.

```java
package com.refactoring.projects.projects;

public class Manager {

    private static final int MESES_DEL_ANIO = 12;
    private static final int BONO_POR_EMPLEADO = 1000;
    private String nombre;
    private double salario;
    private String departamento;
    private int empleadosACargo;

    public void calcularSalarioAnual() {
        System.out.println(getSalario() * MESES_DEL_ANIO + calcularBono());
    }

    private int calcularBono() {
        return getEmpleadosACargo() * BONO_POR_EMPLEADO;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public double getSalario() {
        return salario;
    }

    public void setSalario(double salario) {
        this.salario = salario;
    }

    public String getDepartamento() {
        return departamento;
    }

    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }
```

Foto usando la refactorización de Change Method.

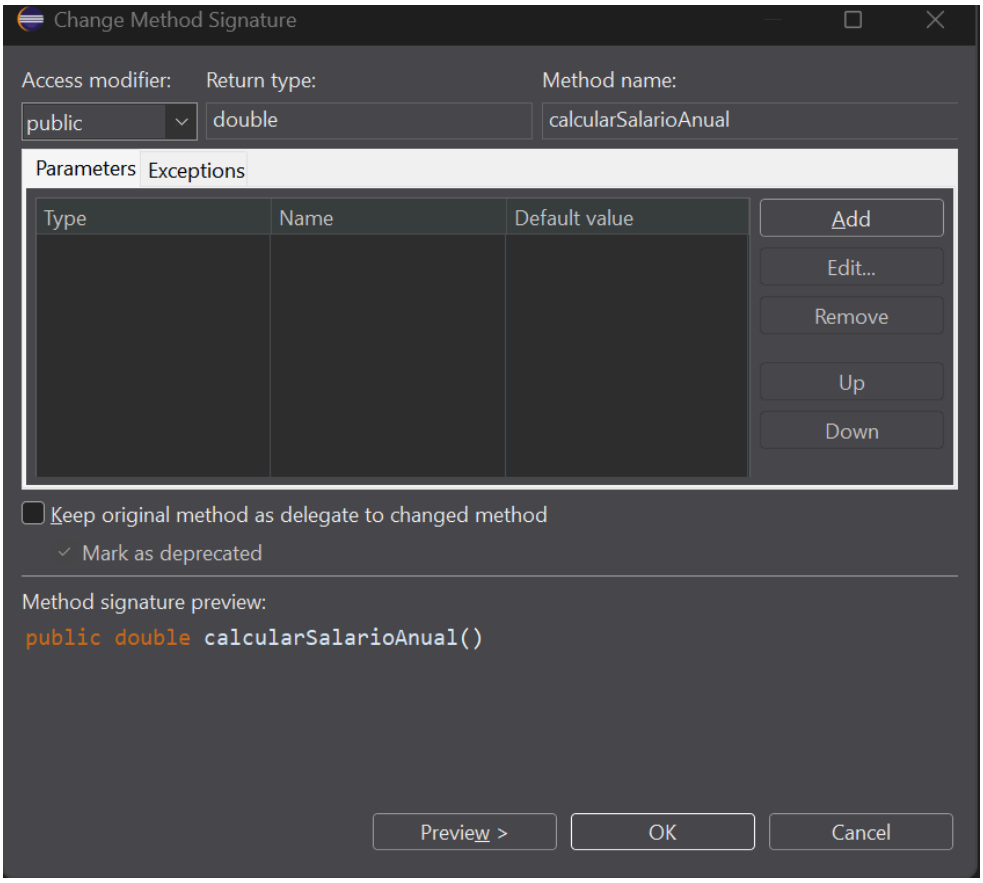


Foto resultado del uso del método.

```
public double calcularSalarioAnual() {
    return getSalario() * MESES_DEL_ANIO;
}
```

Foto de código antes de mover de clase en Employee.

```java
package com.refactoring.projects.project03;

public class Employee {

    private static final int MESES_DEL_ANIO = 12;
    private String nombre;
    private double salario;
    private String departamento;

    public void calcularSalarioAnual() {
        System.out.println(getSalario() * MESES_DEL_ANIO);
    }

    public String getDepartamento() {
        return departamento;
    }

    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }

    public double getSalario() {
        return salario;
    }

    public void setSalario(double salario) {
        this.salario = salario;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Foto usando la refactorización de Change Method.

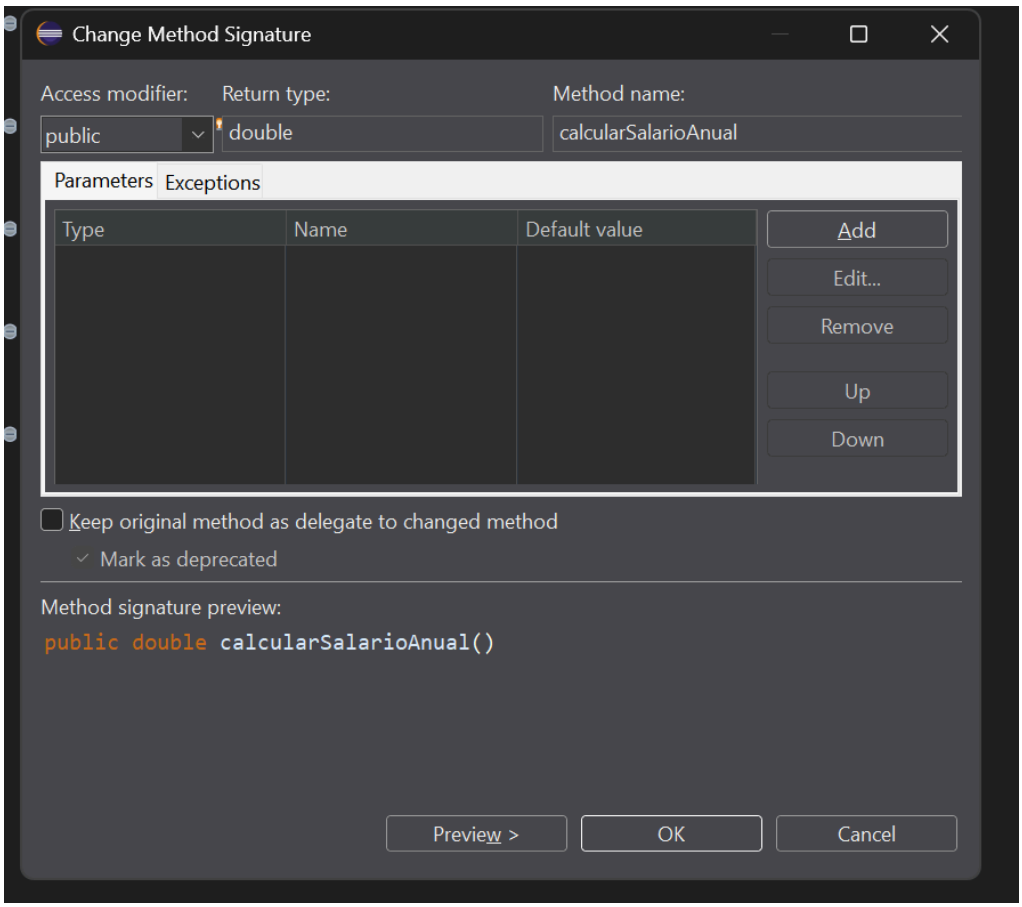


Foto resultado del uso del método.

```java
public double calcularSalarioAnual() {
    return getSalario() * MESES_DEL_ANIO;
}
```

Principal clase EmployeeManager.

Se ha aplicado el patrón de diseño donde las clases de entidad (Employee y Manager) solo gestionan lógica y datos, mientras que la salida por consola se centraliza en EmployeeManager. Esto permite que el sistema sea escalable; por ejemplo, si necesitáramos una interfaz gráfica en el futuro, no tendríamos que modificar las clases de lógica base.

EmployeeManager.java

```java
package com.refactoring.projects.project03;

public class EmployeeManager {

    public static void main(String[] args) {
        // 1. Creamos un Empleado normal
        Employee emp = new Employee();
        emp.setNombre("Juan Pérez");
        emp.setSalario(2000.0);
        emp.setDepartamento("Ventas");

        // 2. Creamos un Manager
        Manager mgr = new Manager();
        mgr.setNombre("Ana López");
        mgr.setSalario(3500.0);
        mgr.setDepartamento("IT");
        mgr.setEmpleadosACargo(5);

        // 3. Imprimimos los resultados (Única clase con System.out)
        System.out.println("--- DATOS DE EMPLEADOS ---");
        imprimirDatos(emp.getNombre(), emp.calcularSalarioAnual());
        imprimirDatos(mgr.getNombre(), mgr.calcularSalarioAnual());
    }

    // Método auxiliar para evitar repetir código de impresión
    private static void imprimirDatos(String nombre, double salarioAnual) {
        System.out.println("Empleado: " + nombre + " | Salario Anual: " + salarioAnual + "€");
    }
}
```

```
<terminated> EmployeeManager [Java Application] C:\Program Files\Java\jdk-2
--- DATOS DE EMPLEADOS ---
Empleado: Juan Pérez | Salario Anual: 24000.0€
Empleado: Ana López | Salario Anual: 47000.0€
```